

REPUBLIQUE ALGERIENNE DEMOCRATIQUE ET POPULAIRE
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE
ECOLE NATIONALE SUPERIEURE D'INFORMATIQUE

CPI 1 **Année 2021 / 2022**

TP
EN ALGORITHMIQUE ET STRUCTURES DE DONNEES DYNAMIQUES (ALSDD)

REALISE PAR :
ARROUCHE Abdellah

Section : D

Groupe N° : 11

Semestre 2

Table de Matières

I. **Introduction** 3
II. **Solution proposée** 4
1. Structures de données 4
2. Les fonctions et procédures utilisées 6
3. Etude Comparative 9
4. Discussion 11
III. **Conclusion** 12

I. Introduction

Dans l'intérêt d'améliorer le stockage des données, les chercheurs en informatique ont inventé des structures dynamiques pour utiliser effectivement un espace mémoire donné et pour compenser les lacunes et inconvénients des structures statiques.

Dans le TP1, il nous demande de proposer une solution pour sauvegarder 1000 identifiants uniques composés de 16 caractères alphanumériques dans des listes linaires chainées de différents types (circulaire unidirectionnelle, circulaire bidirectionnelle, en contiguë, chainage XOR), en définissant sur eux les opérations de création et affichage d'une liste ainsi que l'insertion, suppression et recherche d'un élément.

Au premier lieu, on vise à stocker et manipuler ces identifiants dans une LLC qu'on considère comme une base de données (Liste ordonnée ou on peut insérer, supprimer et rechercher des identifiants), et plus abstraitement on souhaite comparer entre des différents types de liste selon leur efficacité quant à le traitement du premier objectif.

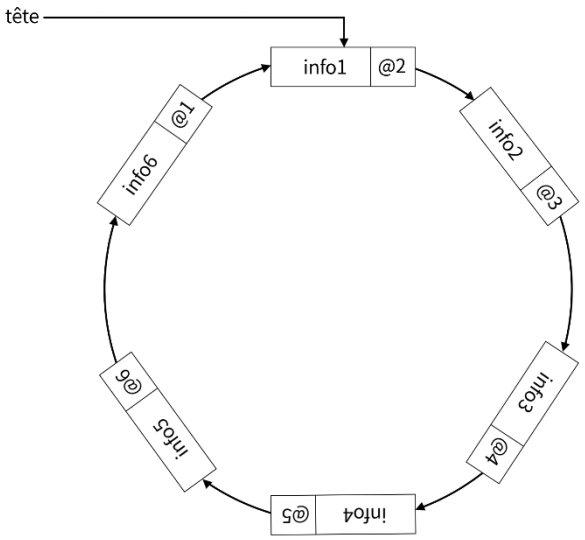
À la suite, on définira les structures qui seront utilisés d'une manière générale (et les adaptés au sujet), les machines abstraites de chaque représentation ainsi que les fonctions et procédures qui lui correspondent, celles qui sont général et celles dédiées à manipuler les identifiants dans la liste, en entamant par une étude comparative élaborée et la conclusion extrapolée de celle-ci.

II. Solution proposée

1. Structures de données

Liste Linaire chainée circulaire unidirectionnelle :

Une liste linaire chainée ou le dernier maillon pointe le premier maillon. On définit la tête de la liste comme point d'accès. Dans une LLC circulaire non ordonnée, n'importe quel maillon peut être considérer comme la tête de la liste.



Le maillon d'une LLC circulaire unidirectionnelle est une structure à 2 champs :

{

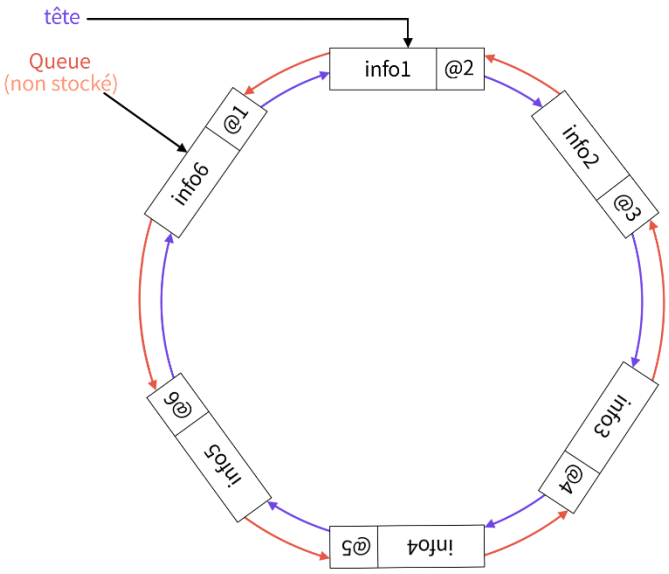
Type quelconque val

Pointeur vers un maillon adr (pointe le maillon suivant)

}

Liste Linaire chainée circulaire bidirectionnelle :

Une liste linaire chainée bidirectionnelle (Chaque maillon pointe le maillon suivant et le maillon précédent) ou le dernier maillon pointe le premier maillon et vice-versa. On définit la tête et la queue de la liste comme point d'accès. Dans une LLC circulaire bidirectionnelle non ordonnée, n'importe quel maillon peut être considérer comme la tête de la liste. De plus, on n'est pas obligé de garder la queue de la liste, elle est récupérée à partir de la tête (La queue ce n'est rien que l'adresse du maillon précédent du maillon pointé par la tête).



Le maillon d'une LLC circulaire bidirectionnelle est une structure à 3 champs :

{

Pointeur vers un maillon adrd (pointe le maillon précédent)

Type quelconque val

Pointeur vers un maillon adrg (pointe le maillon suivant)

}

Liste Linaire chainée en contigüe :

Un liste linaire chainée sous forme d'une matrice mixte, ou le premier champ contient l'information stocké et le deuxième champ :

- Contient l'adresse du maillon suivant si c'est un nombre positif.
- Indique que c'est la fin de la liste si c'est un nombre négatif (généralement -1).

Le maillon d'une LLC en contigüe est une structure à 2 champs :

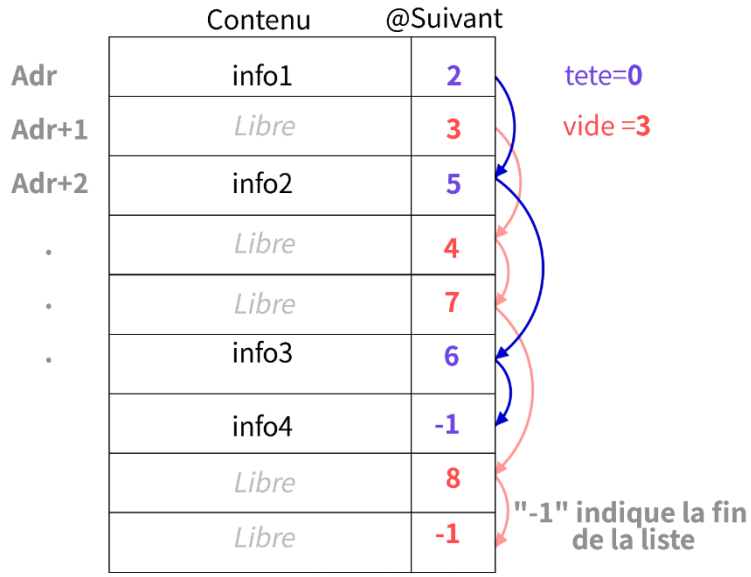
{

Type quelconque val

Entier adr (information sur le maillon courant/maillon suivant)

}

La liste est définie par un vecteur des maillons (vecteur de taille fixe déjà précisé).



Liste Linaire chainée bidirectionnelle avec un seul champ de pointeur :

Il existe plusieurs LLC de ce type (en utilisant l'addition, la soustraction, l'XOR), dorénavant, en considère le chainage XOR.

La liste linaire chainée XOR est une liste linéaire chainée bidirectionnelle avec un seul champ d'adresse au lieu de deux. On stocke dans le champ d'adresse le ou exclusif (XOR) de l'adresse du maillon précédent et l'adresse du maillon suivant ($ptr1 \oplus ptr2$).

Les propriétés mathématiques de l'XOR (Notamment ***Si*** $A \oplus B = C$ ***alors*** $A = B \oplus C$ *et* $B = A \oplus C$) nous permet de :

- Parcourir la liste dans le sens directe à partir de l'adresse du maillon précédent (ptr prec) et le contenu du champ d'adresse du maillon courant (ptr prec XOR ptr suiv), leur ou exclusif nous donne l'adresse du maillon suivant.
- Parcourir la liste dans le sens inverse à partir de l'adresse du maillon suivant (ptr suiv) et le contenu du champ d'adresse du maillon courant (ptr prec XOR ptr suiv), leur ou exclusif nous donne l'adresse du maillon précédent.

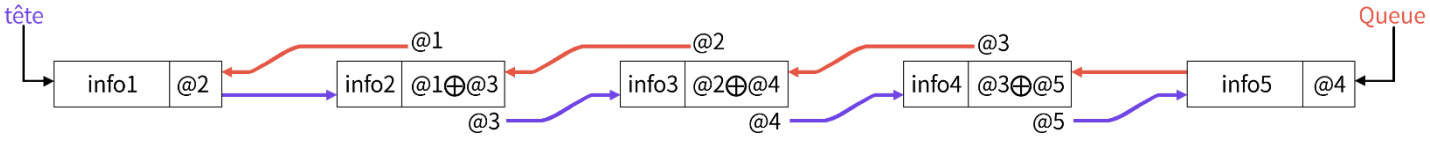
Le maillon d'une LLC circulaire unidirectionnelle est une structure à 2 champs :

{

Type quelconque val

Pointeur vers un maillon adr (contient adresse du maillon précédent XOR adresse du maillon suivant)

}



Les structures utilisées sont définies d'une manière générale. Pour notre solution on donne au champ val le type "pointeur vers un caractère", pour lui affecter l'adresse mémoire de l'identifiant qu'on souhaite stocker.

2. Les fonctions et procédures utilisées

Liste Linaire chainée circulaire unidirectionnelle :

Struct maillon_lc {

Char * val ;

Struct maillon_lc *adr ; } ;

Typedef Struct maillon_lc *ptr_lc ;

- **Void** Allouer_Lc **(ptr_lc** *P**)**
- **Void** Liberer_Lc **(ptr_lc** P**)**
- **Void** Aff_val_Lc **(ptr_lc** P**, char *** Value**)**
- **Void** Aff_adr_Lc **(ptr_lc** P**, ptr_lc** Q**)**
- **Char *** Valeur_Lc **(ptr_lc** P**)**

- **ptr_lc** Suivant_Lc **(ptr_lc** P**)**

- **Void** Create_LLC_c **(ptr_lc** *P**, entier** *n**)**
- **Void** Affiche_liste_c **(ptr_lc** P**)**

- **Void** inserer_element_c **(ptr_lc** *P**, char *** Valeur**)**
- **Void** supprimer_element_c **(ptr_lc** *P**, char *** Valeur**)**
- **ptr_lc** rechercher_element_c **(ptr_lc** P**, char *** Valeur**)**

Liste Linaire chainée circulaire bidirectionnelle :

Struct maillon_lcb {

Struct maillon_lcb *adrg ;

Char * val ;

Struct maillon_lcb *adrd ; } ;

Typedef Struct maillon_lcb *ptr_lcb ;

- **Void** Allouer_Lcb **(ptr_lcb** *P**)**
- **Void** Liberer_Lcb **(ptr_lcb** P**)**
- **Void** Aff_val_Lcb **(ptr_lcb** P**, char *** Value**)**
- **Void** Aff_adrd_Lcb **(ptr_lcb** P**, ptr_lcb** Q**)**
- **Void** Aff_adrg_Lcb **(ptr_lcb** P**, ptr_lcb** Q**)**
- **Char *** Valeur_Lcb **(ptr_lcb** P**)**
- **ptr_lcb** Suivant_Lcb **(ptr_lcb** P**)**
- **ptr_lcb** Precedent_Lcb **(ptr_lcb** P**)**

- **Void** Create_LLC_cb **(ptr_lcb** *P**, entier** *n**)**
- **Void** Affiche_liste_cb **(ptr_lcb** P**)**

- **Void** inserer_element_cb **(ptr_lcb** *P**, char *** Valeur**)**

- **Void** supprimer_element_cb **(ptr_lcb** *P**, char *** Valeur**)**
- **ptr_lcb** rechercher_element_cb **(ptr_lcb** P**, char *** Valeur**)**

Liste Linaire chainée en contigüe :

Struct maillon_lco {

Char * val ;

ptr suiv ; } ; **Typedef int** ptr ;

- **Void** Allouer_Lco **(ptr_lco** *P**)**
- **Void** Liberer_Lco **(ptr_lco** P**)**
- **Void** Aff_val_Lco **(ptr_lco** P**, char *** Value**)**
- **Void** Aff_adrd_Lco **(ptr_lco** P**, ptr_lco** Q**)**
- **Void** Aff_adrg_Lco **(ptr_lco** P**, ptr_lco** Q**)**
- **Char *** Valeur_Lco **(ptr_lco** P**)**
- **ptr_lco** Suivant_Lco **(ptr_lco** P**)**
- **ptr_lco** Precedent_Lco **(ptr_lco** P**)**

- **Void** Create_LLC_co **(ptr_lco** *P**, entier** *n**)**
- **Void** Affiche_liste_co **(ptr_lco** P**)**

- **Void** inserer_element_co **(ptr_lco** *P**, char *** Valeur**)**
- **Void** supprimer_element_co **(ptr_lco** *P**, char *** Valeur**)**
- **ptr_lco** rechercher_element_co **(ptr_lco** P**, char *** Valeur**)**

Liste Linaire chainée XOR :

Struct maillon_lx {

Char * val ;

Struct maillon_lx *adr ; } ; **Typedef Struct** maillon_lx *ptr_lx ;

- **Void** Allouer_Lx **(ptr_lx** *P**)**
- **Void** Liberer_Lx **(ptr_lx** P**)**
- **Void** Aff_val_Lx **(ptr_lx** P**, char *** Value**)**
- **Void** Aff_adrd_Lx **(ptr_lx** P**, ptr_lx** Q**)**
- **Void** Aff_adrg_Lx **(ptr_lx** P**, ptr_lx** Q**)**
- **Char *** Valeur_Lx **(ptr_lx** P**)**
- **ptr_lx** Suivant_Lx **(ptr_lx** P**)**
- **ptr_lx** Precedent_Lx **(ptr_lx** P**)**

- **Void** Create_LLC_x **(ptr_lx** *P**, entier** *n**)**
- **Void** Affiche_liste_x **(ptr_lx** P**)**

- **Void** inserer_element_x **(ptr_lx** *P**, char *** Valeur**)**
- **Void** supprimer_element_x **(ptr_lx** *P**, char *** Valeur**)**
- **ptr_lx** rechercher_element_x **(ptr_lx** P**, char *** Valeur**)**

3. Etude Comparative

Espace mémoire :

Formule de calcul de l'espace mémoire occupé :

$$\begin{array}{c} Pointeurs\ de : tete, queue\ et\ vide\ (s'ils\ existent) \\ + \\ 1000 * (Taille\ du\ maillon) \\ + \\ 1000 * (Taille\ de\ l'identifiant) \\ = \\ \hline Espace\ mémoire\ requis\ par\ la\ liste \end{array}$$

	Type d'architecture	
	x86 (32 bits)	x64 (64 bits)
LLC circulaire	24004 octets (23 ko et 452o)	32008 octets (31 ko et 264o)
LLC circulaire bidirectionnelle	28004 octets (27 ko et 356o)	40008 octets (39 ko et 76o)
LLC en contigüe	24008 octets (23 ko et 456o)	28008 octets (27 ko et 360o)
LLC XOR	24008 octets (23 ko et 456o)	32016 octets (31 ko et 272o)

Nombre d'opération :

1-Affectations :

	Recherche		Insertion		Suppression	
	Minimum	Maximum	Minimum	Maximum	Minimum	Maximum
LLC circulaire	2999	1501500	1505500		2998	1500499
LLC circulaire bidirectionnelle	2999	1501500	9997	1508497	3997	1503496
LLC en contigüe	2999	1501500	2002998	2003997	2000	1499501
LLC XOR	2005980		2002000	2003998	7000	3503500

2-Comparaisons :

	Recherche		Insertion		Suppression	
	Minimum	Maximum	Minimum	Maximum	Minimum	Maximum
LLC circulaire	1999	500500	999	498501	1000	500500
LLC circulaire bidirectionnelle	1999	500500	999	499500	1000	500500
LLC en contigüe	1999	500500	999	500499	1000	500500
LLC XOR	500500		999	498501	1000	500500

Recherche signifie recherche de 1000 éléments dans une liste à mille éléments.

Insertion signifie insertion de 1000 éléments dans une liste initialement vide.

Suppression signifie suppression de 1000 éléments dans un liste à mille éléments.

On suppose que les éléments à rechercher/supprimer est dans la liste pour faciliter les calculs, sinon le nombre de comparaisons peut être : 1 (élément inférieur à l'élément pointé par la tête), 1000 (élément supérieur au dernier élément), 500 (moyenne de la taille de la liste).

4. Discussion

La liste linaire chainée bidirectionnelle occupe beaucoup plus d'espace à cause d'extra champ d'adresse pour pointer l'élément précédent, ce qui est inconvénient, mais c'est un compromis pour pouvoir exécuter les opérations plus rapidement (moins d'affectations).

La liste linaire chainée circulaire et la liste linaire chainée XOR occupe presque le même espace mémoire, avec le bonus de parcoure bidirectionnelle pour la LLC XOR.

La liste linaire chainée en contigüe est la plus efficace en espace mémoire, mais elle reste une structure statique, par conséquent, il faut prévoir le nombre maximal d'éléments, sinon il faut reconstruire la liste en cas d'addition des éléments initialement non considérées. Plus que ça, le champ suivant sert à faciliter le parcoure de la liste, il fonctionne comme un tableau de comptage, donc on peut l'enlever pour mieux optimiser l'espace occupé (à l'inconvénient de compliquer les opérations sur les LLC en contigüe), mais la liste deviendra un tableau simple.

Le nombre de comparaison entre les identifiants est presque le même entre les différents types de listes pour chaque opération, sauf pour la recherche d'un identifiant. La recherche dans une liste XOR n'est pas optimisée pour des recherches successives en ordre croissants à cause d'une de ses caractéristique (On ne peut pas continuer à parcourir une liste XOR avec un seul pointeur qui ne pointe pas une des deux extrémités). Également pour le nombre d'affectation.

La modification de la tête d'une LLC circulaire unidirectionnelle exige la modification du champ d'adresse du dernier maillon, qui nécessite le parcoure de toute la liste à chaque fois, ce qui augmente le nombre d'affectation par rapport aux autres représentations.

Les opérations sur le chainage XOR nécessitent beaucoup plus d'affectation parce que la modification d'un maillon (insertion et suppression) exige la modification du maillon précédant et du maillon suivant (cette complexité demande un peu plus d'effort pour maintenir le code dans certain cas).

L'insertion d'un élément dans une LLC circulaire bidirectionnelle est la plus efficace grâce au accès rapide à la queue de la liste, tandis que la suppression d'un élément d'une LLC en contigüe est la plus efficace grâce à la continuité d'espace mémoire (accès direct au éléments) et la rapidité des modifications (manipulation des cases d'un tableau et non pas des pointeurs).

III. Conclusion

Dans le but de stocker 1000 identifiants uniques et savoir quelle est la moyenne de sauvegarde la plus approprié, on a défini plusieurs types de liste linaire chainée (circulaire unidirectionnelle, circulaire bidirectionnelle, en contiguë, chainage XOR) et des opérations sur eux (Création et affichage d'une liste, recherche, suppression et insertion d'un élément).

Point de vue complexité, la liste linaire chainée circulaires unidirectionnelle et la liste linaire chainée en contiguë sont plus simples. Mais point de vue fonctionnalité, la liste linaire chainée circulaires bidirectionnelle et la liste linaire chainée bidirectionnelle avec un seul champ de pointeur sont plus adéquates.

Même si on a trouvé que les listes linaires chainées en contiguë sont les plus efficaces en espace mémoire, les listes linaires chainées LLC bidirectionnelle avec un seul champ de pointeur représentent un bon compromis entre l'espace mémoire requis et les fonctionnalités données (espace mémoire comparable à une LLC ordinaire et fonctionnalités comparables à celle d'une LLC bidirectionnelle).